Time-Delayed Publicly Verifiable Quantum Computation with Classical Verifiers

Ameer Mohammed
ameer.mohammed@ku.edu.kw
Kuwait University
Kuwait City, Kuwait

Aydin Abadi
aydin.abadi@ncl.ac.uk
Newcastle University
Newcastle upon Tyne, United Kingdom

Jaffer Mahdi
jaffer.mahdi@ku.edu.kw
Kuwait Oil Company
Kuwait City, Kuwait

A ARTIFACT APPENDIX

This appendix supports the paper's central claim: for the linear systems studied, Enhanced Hybrid HHL finishes before the time-lock puzzle (TLP) protecting the verification key can be solved. The artifact packages three self-contained Python experiments behind a single entry point and is intended to be reproducible on any basic computer.

A.1 Abstract

A single Python project at the repository root. The entry point `run_experiments.py` selects among three experiments via CLI flags (`--hhl-only`, `--tlp-only`, `--scaling-only`) and writes PNG plots to `visualization/outputs/` plus JSON data files to the working directory. All inputs (linear system, sweep ranges, clock-qubit count) live in the `INPUTS` block at the top of the file . Licensed under Creative Commons Attribution 4.0 International (CC-BY 4.0); see the Zenodo record for the deposited copy. `run_experiments.py`.

A.2 Description & Requirements

Artifact layout.

- `README.md` — overview and CLI reference.
- `run_experiments.py` — entry point and `INPUTS` block.
- `enhanced_hybrid_hhl/` — Enhanced Hybrid HHL solver.
- `timelock_puzzle/` — TLP garbling, generation, solving.
- `crgc_garbled_circuits/` — reusable garbled-circuit primitives.
- `visualization/` — plotting code and PNG outputs.
- `circuits/` — Bristol-format SHA-256 used inside the TLP.
- `requirements.txt` — Python dependencies.

A.2.1 Security, privacy, and ethical concerns. The artifact performs only local, numerical simulations on a Qiskit `AerSimulator` backend. It makes no network calls, does not modify the host system outside the working directory and `visualization/outputs/`. It also does not do anything that might be considered unethical.

A.2.2 How to access. The artifact is permanently archived at Zenodo under DOI `10.5281/zenodo.20751485` (record zenodo.org/records/20751485), licensed under Creative Commons Attribution 4.0 International (CC-BY 4.0). Download the `Quantum_vertification_artifact.zip` from the record to obtain the full repository.

A.2.3 Hardware dependencies. A regular laptop suffices. Reference machine: 2024 Macbook Pro, M3 Max, 36 GB RAM. No GPU, FPGA, QPU, or cloud account is required; all quantum circuits run on the local Qiskit `AerSimulator`. Absolute times will vary with the host CPU, but the qualitative relationships between HHL, TLP, and the paper's claims are hardware-independent.

A.2.4 Software dependencies. Python $\geq$ 3.10 on macOS, Linux, or Windows. The five direct dependencies are pinned in `requirements.txt`: `qiskit` $\geq$ `1.0.0`, `qiskit-aer` $\geq$ `0.14.0`, `numpy` $\geq$ `1.26.0`, `matplotlib` $\geq$ `3.8.0`, `scipy` $\geq$ `1.12.0`. The paper's timings were collected on AWS Braket SV1; this artifact uses Qiskit `AerSimulator` locally. Absolute timings differ across simulators, but the relative ordering is preserved. **Not validated:** the paper's exact Braket SV1 wall-clock numbers (they require paid cloud access); this artifact validates the qualitative relationships, not the absolute Braket times.

A.2.5 Benchmarks. None required. Toy linear systems are used for the experiments; the TLP's sequential function uses the SHA-256 Bristol-format circuit shipped at `circuits/sha256.txt`.

A.3 Set Up

A.3.1 Installation. Using the `uv` package manager (project-preferred):

```
uv sync
```

Equivalently, with `pip`:

```
pip install -r requirements.txt
```

This resolves all dependencies.

A.3.2 Basic test.

```
uv run python run_experiments.py --hhl-only
```

On success the script prints an HHL total time on the order of seconds and writes `visualization/outputs/hhl_results.png`. If the file appears and the graph shows both hhl matrix components, the artifact is ready for the full evaluation below.

A.4 Evaluation Workflow

A.4.1 Major claims.

(C1): *Enhanced Hybrid HHL on the 2×2 system* $A = [[2, 1], [0, 1.5]]$, $b = [1, 2]$ *produces a sign-agnostic (negative values not captured by result) solution with finite L2 error and values close to the classical linear system of eqns solution, completing in a few seconds on the local Qiskit* `AerSimulator`. Proven by (E1); see `visualization/outputs/hhl_results.png`.

(C2): *TLP* `PSolve` *time scales linearly in T (intrinsic to the protocol: a garbled SHA-256 is evaluated sequentially T times).* Proven by (E2); see `tlp_benchmark_plot.png` and `tlp_benchmark_results.json::reveal_times`.

(C3): *For $N \in \{2, 4, 8\}$ with calibrated $T(N) \in \{250, 800, 4000\}$, the projected TLP* `PSolve` *time exceeds the measured HHL average time at every N (the paper's main quantitative claim). Absolute times vary with the host CPU; the TLP > HHL ordering is hardware-independent. As it can be selected and configured by the user, by changing the TLP time parameter T for a given setup* Proven by (E3); see `scaling_results.png` and `scaling_results.json::tlp_solve_times` vs. `hhl_avg_times`.

A.4.2 Experiments.

(E1): *[Enhanced Hybrid HHL, <1 min + <1 min + <1 MB output].* Solves the 2×2 system $A = [[2, 1], [0, 1.5]]$, $b = [1, 2]$ (set by `HHL_A`, `HHL_B`) using Enhanced Hybrid HHL on the local `AerSimulator`.
Preparation: `uv sync` (or `pip install -r requirements.txt`).
Execution: `uv run python run_experiments.py --hhl-only`.
Results: Inspect `visualization/outputs/-hhl_results.png` and the printed `l2_error`, `circuit_depth`, `total_time`. Pass criterion: Each component's value is similar to the classical solution, and the `l2_error` is a small finite value (a small apparent discrepancy is expected since HHL recovers only amplitude magnitudes, not signs).

(E2): *[TLP benchmark, <1min + <2min + <1 MB output].* Times the three TLP phases (garbling, generation, solving) for $T \in \{1, 2, 3, 4\}$ with 3 trials per T (set by `TLP_T_MIN`, `TLP_T_MAX`, `TLP_TRIALS`). Defaults finish in a few minutes on the reference machine.
Preparation: Same as E1.
Execution: `uv run python run_experiments.py --tlp-only`.
Results: Inspect `tlp_benchmark_plot.png` and `tlp_benchmark_results.json`. Pass criteria: all trials pass `recovered == secret_bit`, and `reveal_times` grows linearly with T with a small intercept and error.

(E3): *[Scaling, <1 min + <20 min + <1 MB output].* For each $N \in \{2, 4, 8\}$ (see `SCALING_N_VALUES`), generates 10 random $N \times N$ matrices, runs Enhanced Hybrid HHL on each, calibrates TLP `PSolve` at $T = 1, \ldots, 5$ on this machine, fits a linear model, and projects `PSolve` to the calibrated $T(N) \in \{250, 800, 4000\}$ (see `N_TO_T`). Pass `--tlp-actual` to run the full pipeline at the large $T(N)$ (takes a lot of time depending on T; only needed to verify the linear projection), else you can keep it as is to allow for the program to estimate the projection value based on the linear relationship.
Preparation: Same as E1. Confirm every N in `SCALING_N_VALUES` has an entry in `N_TO_T`.
Execution: `uv run python run_experiments.py --scaling-only`.
Results: Inspect `scaling_results.png` and `scaling_results.json`. Pass criteria: fidelity ≥ 0.85 across all N, and the projected TLP `PSolve` time at $T(N)$ exceeds the HHL time at every N (C3). Absolute times vary with the host CPU.

A.5 Notes on Reusability

The protocol is not HHL-specific, HHL is one example quantum algorithm. `PythonGarbledTLP` generates a TLP for any quantum circuit that produces a classical output, so the same construction works for HHL, VQE, QAOA, Shor's, Grover's, etc. Use cases include verifiability of delegated quantum computation, fair release of a quantum-computation result on a cloud backend and so on.

Two parameters can be altered for different use cases:

- **TLP time parameter t (or T).** Set to the value that makes `PSolve` just exceed the quantum-computation time on the target hardware. Use `--tlp-only` to measure the per-unit-t slope on a new machine or with a new wrapped circuit, and `--scaling-only` to project that slope to large t via the linear fit (the linearity is intrinsic to the TLP and holds for any wrapped circuit, not just HHL).
- **Quantum problem instance.** For HHL this is the matrix A and right-hand side b; set `HHL_A` and `HHL_B` at the top of `run_experiments.py` (the system is auto-Hermitianized by `hermitianize_matrix`). For other algorithms, replace the call inside `run_scaling_experiment` with the algorithm's own estimate/measurement routine.

Further optional customizations: swap the TLP's sequential function for different security models (SHA-256 default, lattice-based in `timelock_puzzle/lattice_sequential_function.py`, POSW in `timelock_puzzle/posw_lattice.py`; custom functions implement the `SequentialFunction` interface in `timelock_puzzle/-sequential_functions.py`), and substitute the dependencies (HHL or CRGC libraries) by editing the imports at the top of `run_experiments.py`.

A.6 Version

Based on the LaTeX template for Artifact Evaluation V20220926.